After the client completes it will automatically close the `DtlsDatagramChannel`, which will trigger shutdown of the associated `SSLEngine` object. Closing a DTLS session is not as simple as just closing the UDP channel, because each party must send each other a *close-notify* alert message to signal that the DTLS session is being closed. In Java, the process is similar to the handshake loop that you saw earlier in listing 12.2. First, the client should indicate that it will not send any more packets by calling the `closeOutbound()` method on the engine. You should then call the `wrap()` method to allow the engine to produce the close-notify alert message and send that message to the server, as shown in listing 12.4. Once the alert has been sent you should process incoming messages until you receive a corresponding close-notify from the server, at which point the `SSLEngine` will return true from the `isInboundDone()` method and you can then close the underlying UDP `DatagramChannel`.

If the other side closes the channel first then the next call to `unwrap()` will return a `CLOSED` status. In this case you should reverse the order of operations: first closing the inbound side and processing any received messages and then closing the outbound side and sending your own close-notify message.

Listing 12.4 Handling shutdown

```
@Override
public void close() throws IOException {
    sslEngine.closeOutbound();    #A
    sslEngine.wrap(appData.flip(), sendBuf);    #B
    appData.compact();    #B
    channel.write(sendBuf.flip());    #B
```